

Cooking Team Feat

팀원 : 채현후, 홍성민, 권택현



목차

01 프로젝트 개요

02 팀 구성 및 역할

03 프로젝트 일정

04 프로젝트 아키텍처

05 기능 소개

06 트러블 슈팅



프로젝트 개요



프로젝트 선정 이유

많은 요리 앱은 단순 정보 제공에 그쳐서 재미나 참여 유도 요소가 부족, 그래서 요리에 재미를 더해보자는 취지와 요리를 처음 도전하는 사람에게는 친근하고 쉬운 접근성을 제공하고 싶어 선정

프로젝트 주제

요리를 어려워하는 사람들도 손쉽게 도전하고, 자신만의 레시피를 공유하며 요리에 대한 재미와 성취감을 느낄 수 있는 웹/앱 플랫폼



팀 구성 및 역할



채현후

팀장, FrontEnd

UI/UX 디자인
반응형 UI 컴포넌트 최적화
도감 페이지 기능
내 게시글 페이지 기능
도전과제, 칭호, 랭킹 페이지 기능
프로필 기능
밥풀상점(결제 모듈 연동)

홍성민

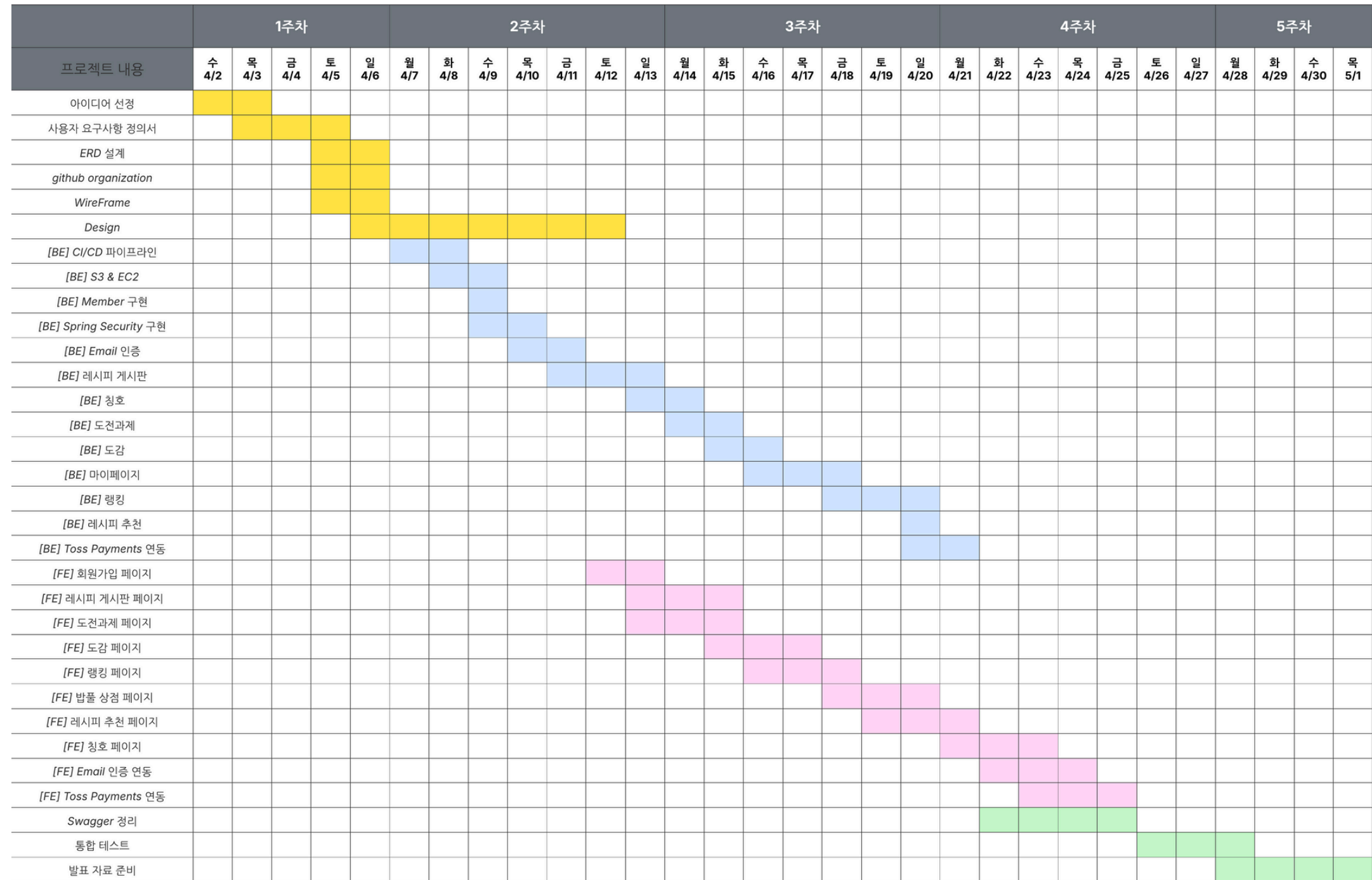
팀원, FrontEnd

UI/UX 디자인, 컴포넌트 작업
Email 인증 연동 작업
회원가입 페이지 기능
로그인, 로그아웃 페이지 기능
프로필 기능
비밀번호 변경 기능
회원탈퇴 기능

권택현

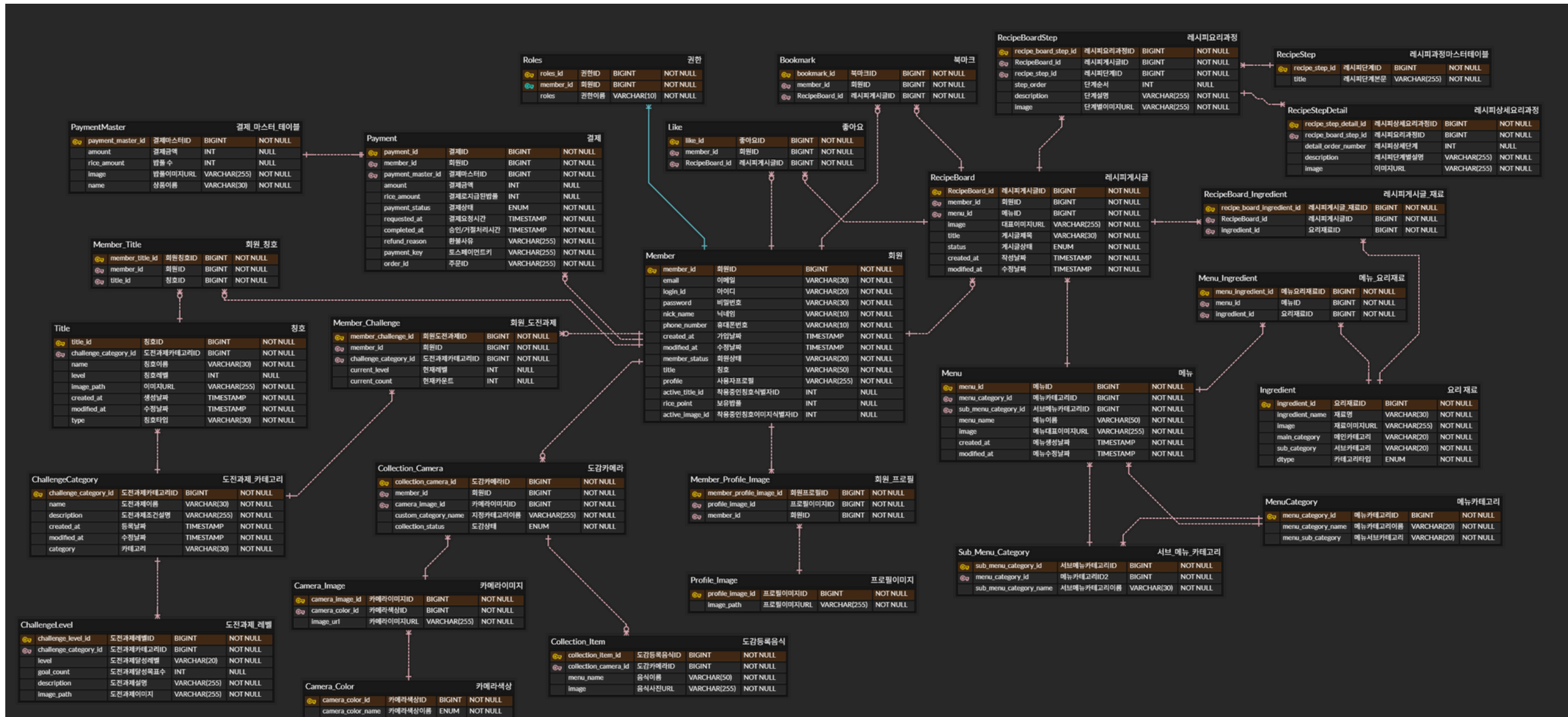
팀원, BackEnd

AWS S3 이미지 업로드
CI/CD 파이프라인 구축
Email 인증 시스템
Spring Security를 통한 인증/인가 처리
Redis를 이용한 로그인 상태, 이메일 인증 관리
Toss 결제 API를 이용한 결제 시스템
백엔드 기능 구현



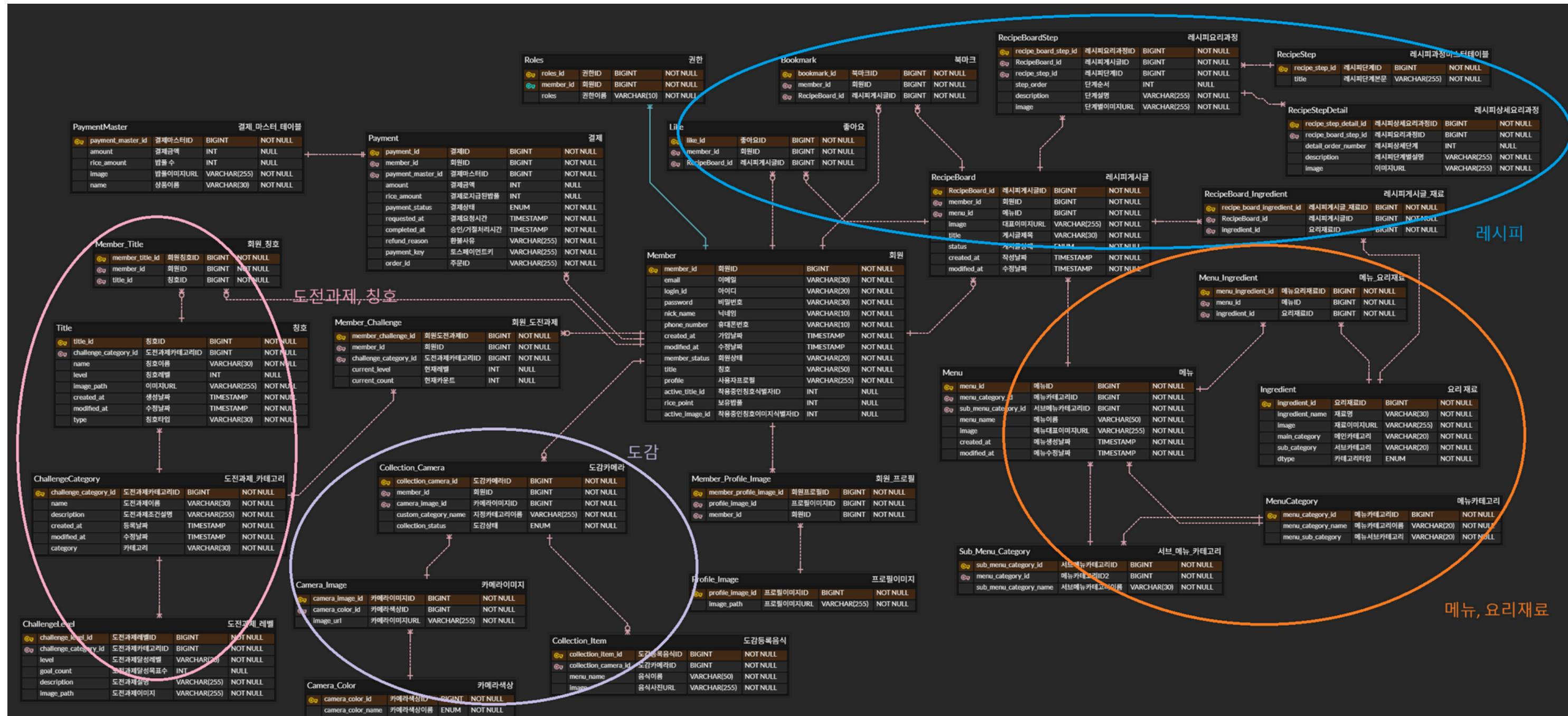


ERD



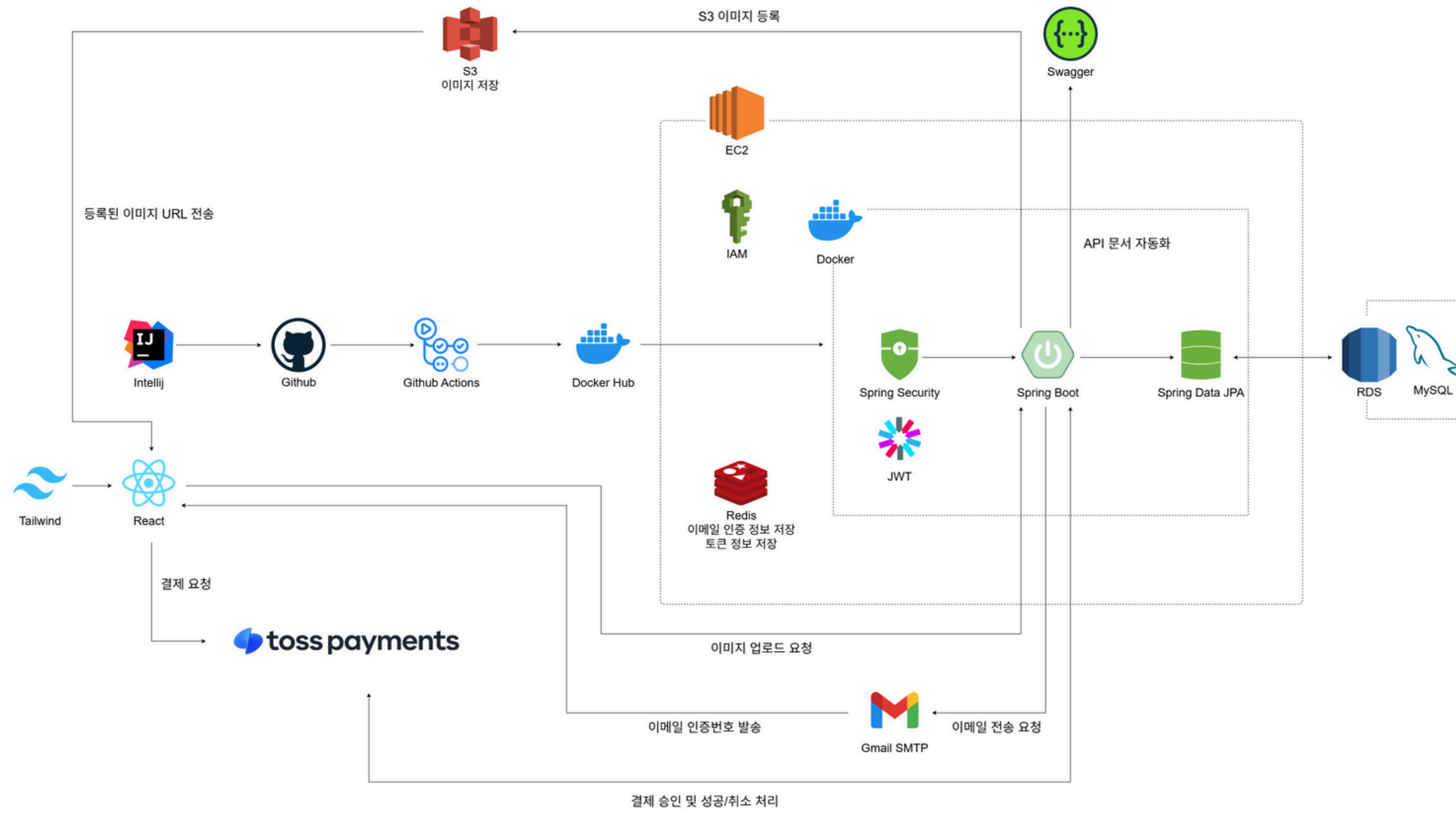


ERD





프로젝트 아키텍처





API 문서



회원 컨트롤러

회원 관련 컨트롤러



POST

/members 회원 가입



DELETE

/members 회원 탈퇴(자신)



PATCH

/members 회원 수정(닉네임)



POST

/members/phone 핸드폰번호 유효성 검증



POST

/members/name 닉네임 중복 검증



POST

/members/id 아이디 중복 검증



POST

/members/findid 아이디 찾기



PATCH

/members/password 비밀번호 재설정



GET

/members/{member-id} 회원 조회





기능 소개



시연

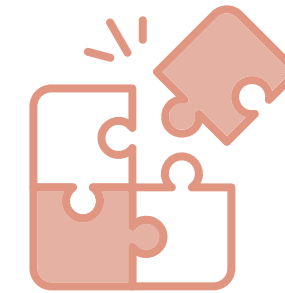


트러블 슈팅



문제 발생

- MemberService에서 CollectionCameraService를 주입받아 기본 도감 생성을 시도
- CollectionCameraService에서는 회원 검증을 위해 MemberService 주입 받은 상태
- 이로 인해 Spring Bean 생성 시 순환 참조 문제 발생



문제 해결

- @Lazy 어노테이션 사용해 순환 참조 회피도 고려했지만 내부적으로는 의존성이 남아 있어 결합도 높음
- 서비스 간 참조 방식 보다 도메인 중심 흐름으로 기능 직접 구현
- 기본 도감 생성 로직을 내부에서 직접 세팅하는 방식으로 문제 해결



트러블 슈팅



```
*****
APPLICATION FAILED TO START
*****

Description:

The dependencies of some of the beans in the application context form a cycle:

    collectionCameraController defined in file [C:\backend\build\classes\java\main\com\springboot\collectioncamera\controller\CollectionCameraControl
    ┌
    | collectionCameraService defined in file [C:\backend\build\classes\java\main\com\springboot\collectioncamera\service\CollectionCameraService.class]
    ↑   ↓
    | memberService defined in file [C:\backend\build\classes\java\main\com\springboot\member\service\MemberService.class]
    └
```



트러블 슈팅



```
private final CollectionCameraService collectionCameraService;  
this.collectionCameraService = collectionCameraService;
```

```
// Member 저장  
Member savedMember = memberRepository.save(member);  
  
collectionCameraService.createDefaultCollection(savedMember);  
  
return savedMember;
```

```
// Member 저장  
Member savedMember = memberRepository.save(member);  
  
// 기본 도감 카메라 세팅 (직접)  
CameraImage defaultCameraImage = cameraImageRepository.findById(1L)  
    .orElseThrow(() -> new BusinessException(ExceptionCode.CAMERA_IMAGE_NOT_FOUND));  
  
CollectionCamera defaultCollection = new CollectionCamera();  
defaultCollection.setCustomCategoryName("도감 예시");  
defaultCollection.setCameraImage(defaultCameraImage);  
defaultCollection.setMember(savedMember);  
defaultCollection.setCollectionStatus(CollectionCamera.CollectionStatus.PUBLIC);  
  
collectionCameraRepository.save(defaultCollection);
```

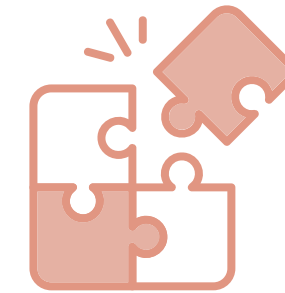


트러블 슈팅



문제 발생

- 사용자가 인증번호 전송/재전송 시, 타이머가 제대로 종료되지 않아 중복 실행되는 문제가 발생
- 타이머가 리셋되지 않고 이어서 동작하거나, 두 개 이상의 타이머가 동시에 돌아감
- 그로 인해 카운트다운 숫자가 겹쳐 보이거나, 시간이 비정상적으로 빨리 0이 되는 등 UI/UX에 혼란 발생



문제 해결

- 인증번호 전송/재전송 시마다 타이머를 완전히 초기화하고, 새로운 타이머만 동작하도록 로직 수정
- timerKey, timerActive 등의 state를 활용해 컴포넌트를 강제로 리렌더링, 이전 타이머는 제거됨
- 인증번호 재전송 시, 입력 필드와 안내 메시지도 초기화하여 사용자 혼동 방지



트러블 슈팅



```
// 인증번호 전송/재전송 버튼 클릭
const handleSendVerification = async (e) => {
  await sendVerificationCode(fullEmail);
  setIsVerificationSent(true);
  setVerificationDisabled(false);
  setTimerActive(true); // 이전 타이머가 종료되지 않아 중복 실행
};
```

```
// (해결) 타이머를 완전히 초기화 후 재시작
const handleSendVerification = async (e) => {
  await sendVerificationCode(fullEmail);
  setIsVerificationSent(true);
  setVerificationDisabled(false);
  setTimerActive(false); // 기존 타이머를 먼저 종료
  setTimerKey(prev => prev + 1); // key를 변경해 컴포넌트 리렌더링(타이머 완전 초기화)
  setTimeout(() => setTimerActive(true), 0); // 다음 tick에 타이머 시작
  setVerificationCodeInput(''); // 인증번호 입력값 초기화
  setErrors((prev) => ({ ...prev, verificationCode: "" }));
  setSuccess((prev) => ({ ...prev, verificationCode: "" }));
};
```

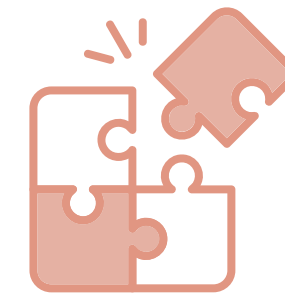


트러블 슈팅



문제 발생

- 토큰 만료 시 자동 로그아웃이 되지 않음 → 사용자가 직접 로그아웃해야 하는 불편함 존재
- API 요청마다 개별적으로 401 에러 처리가 필요 → 코드 중복 발생
- 중앙 집중식 관리 부재로 유지보수 어려움
- 사용자 경험의 일관성 부족, 만료된 토큰으로 계속 요청이 가능해 보안 취약성도 존재



문제 해결

- Axios 인터셉터를 통해 중앙 집중식 토큰 관리 구현
- accessToken, memberId, userInfo 등 민감 데이터 정리, /login 페이지로 리다이렉트
- 사용자에게 토스트 메시지로 알림 제공 → UX 개선
- 모든 요청에 자동으로 토큰 추가, 모든 응답에서 401 에러 발생 시 자동 로그아웃 처리



트러블 슈팅



```
// 문제가 발생했던 코드
const instance = axios.create({
  baseURL: process.env.REACT_APP_API_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});
// 토큰 만료 시 사용자가 직접 로그아웃해야 함
instance.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // 사용자에게 토큰 만료 알림만 표시
      alert('세션이 만료되었습니다. 다시 로그인해주세요.');
```

```
// 요청 인터셉터 - 모든 요청에 토큰 추가
instance.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('accessToken');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// 응답 인터셉터 - 토큰 만료 처리
instance.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // 토큰 만료 시 자동 로그아웃 처리
      localStorage.removeItem('accessToken');
      localStorage.removeItem('memberId');
      localStorage.removeItem('userInfo');

      // 로그인 페이지로 리다이렉트
      window.location.href = '/login';

      // 사용자에게 토스트 메시지 표시
      showToast('세션이 만료되었습니다. 다시 로그인해주세요.', 'error');
    }
    return Promise.reject(error);
  }
);
```



질문과 답변



Q & A



프로젝트 회고



회고



감사합니다

THANK
You!